Republic of the Philippines
BOHOL ISLAND STATE UNIVERSITY
Cogtong, Candijay, 6312, Bohol, Philippines
Office of the College of Sciences
Balance | Integrity | Stewardship | Uprightness

BAGONG PILIPINAS
TÜVRheinland CERTIFIED
Management System ISO 9001:2015
www.tuv.com ID 9108658239

PROJECT IN DATA STRUCTURES AND ALGORITHMS

MAZE ESCAPE: THE ADVENTURE GAME

Submitted by:

Edrian Galvez
JV Kristian Jamisola
Gian Salvador
Ian Galve
Nelson Bajao

BSCS 2B

Submitted to:

ALMA MAE B. AUXTERO

Instructor

Republic of the Philippines
BOHOL ISLAND STATE UNIVERSITY
Cogtong, Candijay, 6312, Bohol, Philippines
Office of the College of Sciences
Balance | Integrity | Stewardship | Uprightness

BAGONG PILIPINAS
TÜVRheinland CERTIFIED
Management System ISO 9001:2015
www.tuv.com ID 9108658239

Date submitted: November 4, 2026

1 Game Mechanics or System Features

Purpose and users

Maze Escape: The Adventure Game is a Java desktop game designed for players and students who want to explore a maze while seeing how data structures and algorithms can be used in a working application. The player's goal is to travel from the starting point to the exit without crossing walls. The project focuses on one fixed 15 × 15 hard-mode maze and demonstrates movement, undo, and shortest-path assistance.

How the application works

The game opens with a main menu where the player can start Hard Mode, view the instructions, or exit the application. During gameplay, the player moves with the arrow keys. A move is accepted only when the destination cell is inside the maze and is not a wall. The move counter updates as the player travels. Pressing U restores the previous player position, H displays a shortest-path hint, and R restarts the maze. The player wins by reaching the enchanted exit, after which a completion dialog displays the difficulty and total moves.

Main features

• Main menu and instructions — provide the game options and explain the controls.
• Hard Mode maze — presents a 15 × 15 maze containing walls, walkable paths, and an exit.
• Keyboard movement and wall collision — arrow keys move the player through valid path cells; walls block movement.
• Move counter — displays the number of successful moves made during the game.
• Undo (U) — returns the player to the previous position using the custom Stack.
• Shortest-path hint (H) — uses Breadth-First Search (BFS) and the custom Queue to show a suggested route.
• Restart (R) and Main Menu — allow the player to restart the maze or return to the menu.
• Victory dialog — appears after the player reaches the exit and displays the total moves.

2 Data Structures and Algorithms Used

The game uses three main data structures: a two-dimensional array, a custom Stack, and a custom Queue. The Stack and Queue are implemented manually with linked nodes and their basic operations rather than relying on built-in collection classes. Breadth-First Search (BFS) is the main path-finding algorithm.

Data structure 1 — Two-Dimensional Array

Purpose Stores the maze layout, where 0 represents a wall, 1 represents a walkable path, and 2 represents the exit. Related two-dimensional arrays store visited information and parent positions while BFS searches.
Implementation Implemented as Java int[][] arrays in MazeEscape.java.
Operations and algorithms The game reads maze[row][column] to check a destination cell. BFS also uses visited, parentRow, and parentCol arrays to track explored cells and rebuild the route to the exit.
Reason for selection An array provides direct access to a cell by row and column, which suits a grid-based maze. Its size is fixed, so changing the maze dimensions requires changing the array.

Data structures 2 and 3

Data structure 2 — Custom Stack
Purpose Stores the player's previous positions so the Undo feature can return to the last position.
Implementation Manually implemented in MazeEscape.java using linked Node objects and a top reference.
Operations and algorithms push adds a previous position, pop removes and returns the most recent position, and isEmpty checks whether there is a position to undo. This follows Last In, First Out (LIFO).
Reason for selection A Stack matches undo because the latest move must be reversed first. A linked implementation can grow as moves are made, but each operation requires node management and uses memory for links.

Data structure 3 — Custom Queue
Purpose Holds maze positions waiting to be explored by the BFS path-finding algorithm.
Implementation Manually implemented in MazeEscape.java using linked Node objects with front and rear references.
Operations and algorithms enqueue adds a position at the rear, dequeue removes a position from the front, and isEmpty checks whether the queue has items. This follows First In, First Out (FIFO).
Reason for selection A Queue supports BFS because positions are explored in the order they are discovered. A linked queue can grow as needed, but it requires link management and memory for nodes.

Algorithm — Breadth-First Search (BFS)
BFS explores reachable neighboring cells level by level using the custom Queue. It marks cells as visited and records each cell's parent row and column. When the exit is found, the parent information is followed backward to reconstruct the shortest path in an unweighted maze. The hint feature displays this route as the golden trail.

3 System Screenshots

The following figures show the actual running Java application and its main gameplay features.

Main interface — Main Menu

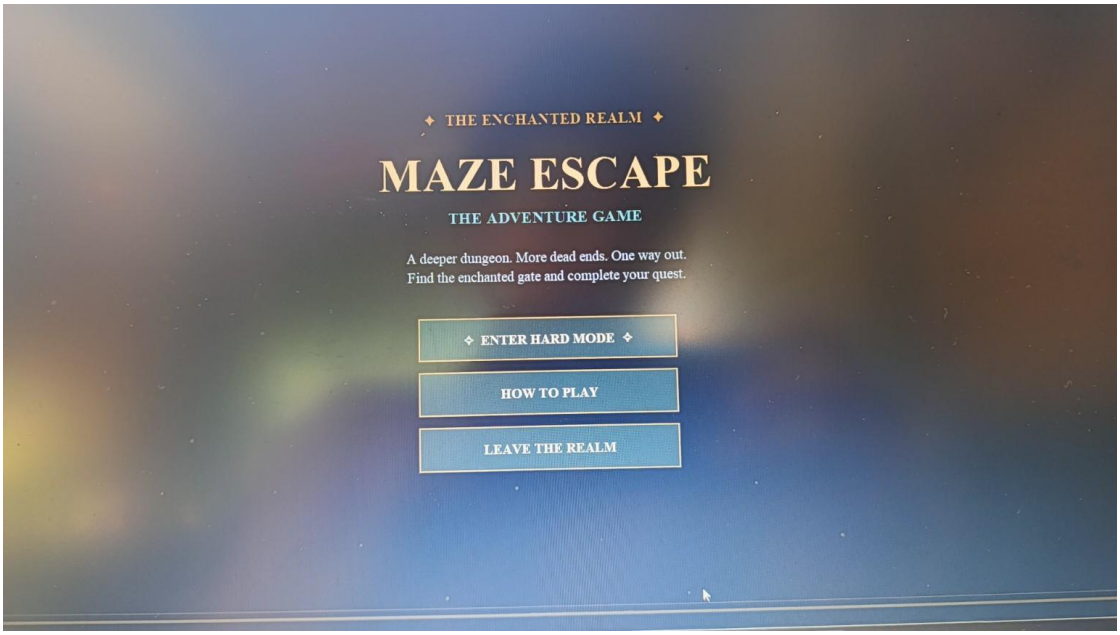


Figure 1. Main menu of Maze Escape: The Adventure Game. The player can enter Hard Mode, view the instructions, or leave the game.

Gameplay — Hard Mode

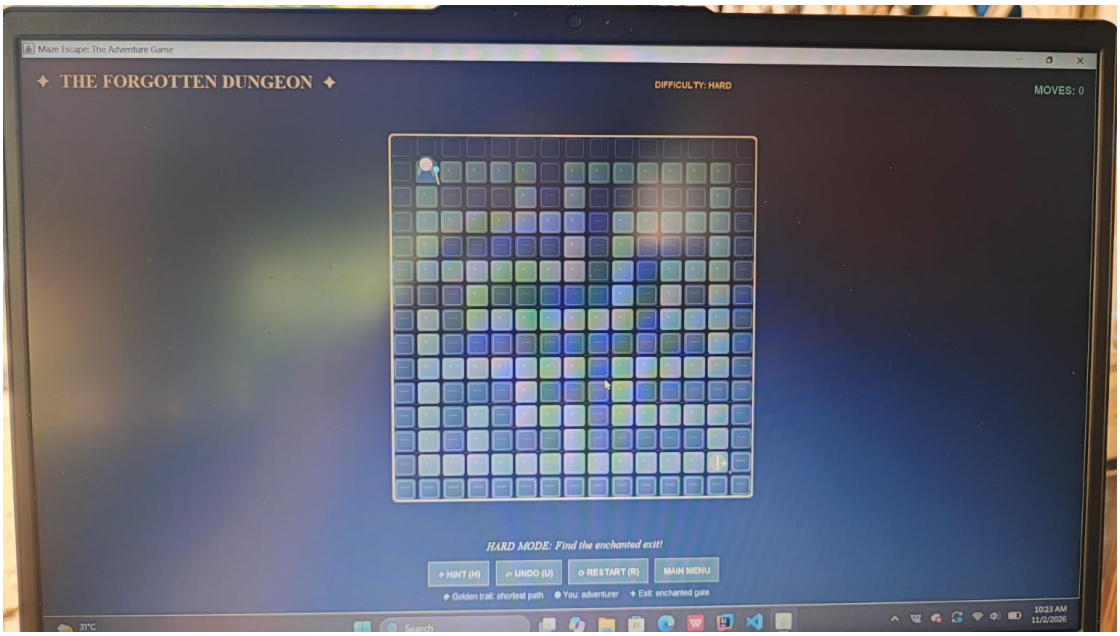


Figure 2. Hard Mode gameplay screen showing the 15 × 15 maze, player position, difficulty indicator, move counter, and available controls.

Important feature — BFS Shortest-Path Hint

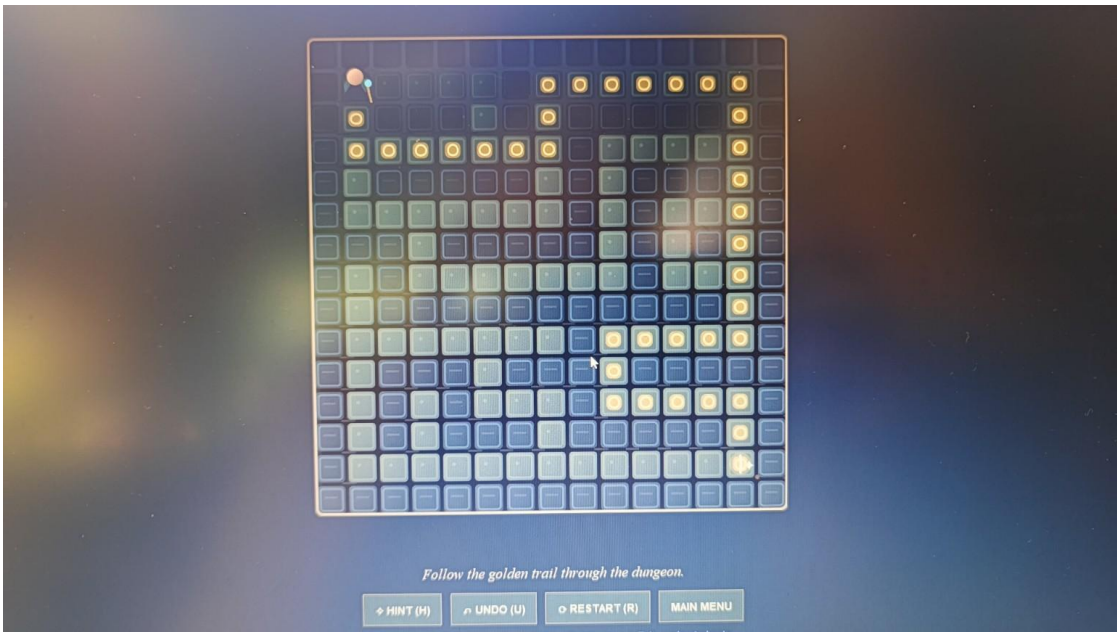


Figure 3. The golden trail shows the route found by Breadth-First Search. The BFS process uses the custom Queue and visited/parent arrays.

Actual output — Victory Screen

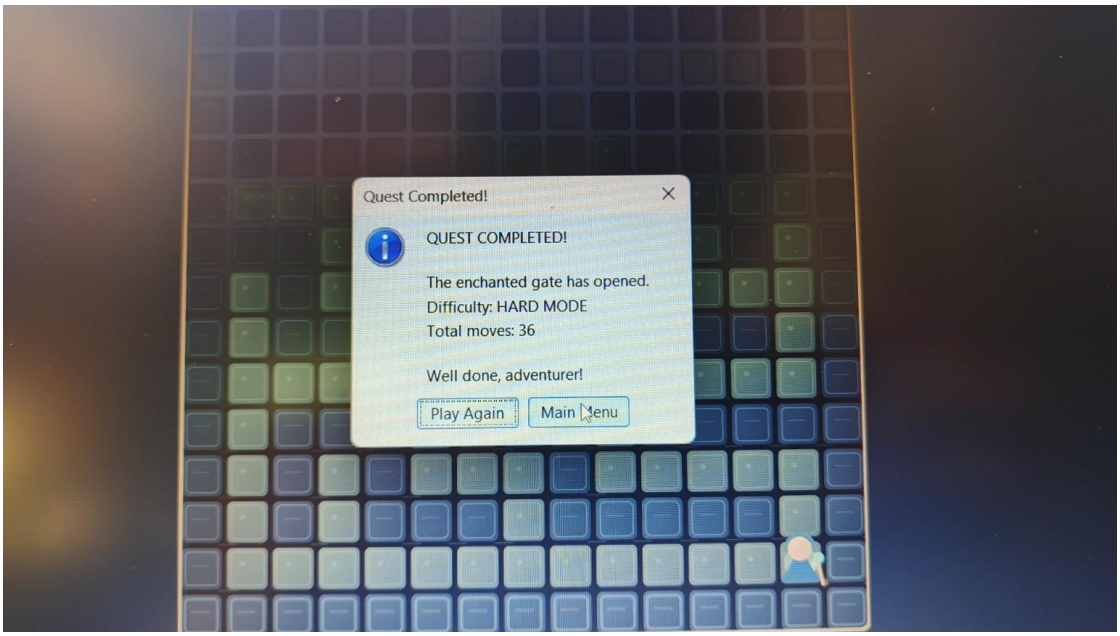


Figure 4. Victory dialog displayed after the player reaches the enchanted exit. It confirms completion and reports the difficulty and total moves.